COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18/2026	**DATE SUBMITTED:**	09/18/2026
NAME:	Moradillo, Eunice	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 9.1 **Tree ADT**

1. PROCEDURE OUTPUT

Table 9-1: General Tree

CODE SNIPET (header)

```
#ifndef GENERALTREE_H
#define GENERALTREE_H

#include <iostream>

template <typename T>
class GeneralTree {
private:
    T key;
    GeneralTree<T>* firstChild;
    GeneralTree<T>* nextSibling;

public:
    //constructor
    GeneralTree(T rootObj);

    //inserting a child node, gets added after the existing ones and returns the new node
    GeneralTree<T>* insertChild(T newNode);

    GeneralTree<T>* getFirstChild();
    GeneralTree<T>* getNextSibling();

    T getRootVal();
    void setRootVal(T obj);

    bool isLeaf();

    //just to check the tree looks right
    void printTree(int depth = 0);
};

template <typename T>
GeneralTree<T>::GeneralTree(T rootObj) {
    key = rootObj;
    firstChild = nullptr;
    nextSibling = nullptr;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::insertChild(T newNode) {
    GeneralTree<T>* newChild = new GeneralTree<T>(newNode);

    if (firstChild == nullptr) {
        //no kids yet so this is the first one
        firstChild = newChild;
    } else {
        //otherwise go to the end of the sibling list and stick it there
        GeneralTree<T>* temp = firstChild;
        while (temp->nextSibling != nullptr) {
            temp = temp->nextSibling;
        }
        temp->nextSibling = newChild;
    }

    return newChild;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getFirstChild(){
    return firstChild;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getNextSibling(){
    return nextSibling;
}

template <typename T>
T GeneralTree<T>::getRootVal() {
    return key;
}
```

```
template <typename T>
GeneralTree<T>* GeneralTree<T>::getFirstChild(){
    return firstChild;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getNextSibling(){
    return nextSibling;
}

template <typename T>
T GeneralTree<T>::getRootVal() {
    return key;
}

template <typename T>
void GeneralTree<T>::setRootVal(T obj) {
    key = obj;
}

template <typename T>
bool GeneralTree<T>::isLeaf() {
    return firstChild == nullptr;
}

template <typename T>
void GeneralTree<T>::printTree(int depth) {
    for (int i = 0; i < depth; i++) {
        std::cout << "  ";
    }
    std::cout << key << std::endl;

    if (firstChild != nullptr) {
        firstChild->printTree(depth + 1); //go down
    }
    if (nextSibling != nullptr) {
        nextSibling->printTree(depth); //next one over
    }
}

#endif
```

CODE SNIPPET (main)

```
int main() {
    // Build the tree exactly as shown in the figure
    GeneralTree<char>* A = new GeneralTree<char>('A');

    GeneralTree<char>* B = A->insertChild('B');
    GeneralTree<char>* C = A->insertChild('C');
    GeneralTree<char>* D = A->insertChild('D');
    GeneralTree<char>* E = A->insertChild('E');
    GeneralTree<char>* F = A->insertChild('F');
    GeneralTree<char>* G = A->insertChild('G');

    GeneralTree<char>* H = D->insertChild('H');

    GeneralTree<char>* I = E->insertChild('I');
    GeneralTree<char>* J = E->insertChild('J');

    GeneralTree<char>* P = J->insertChild('P');
    GeneralTree<char>* Q = J->insertChild('Q');

    GeneralTree<char>* K = F->insertChild('K');
    GeneralTree<char>* L = F->insertChild('L');
    GeneralTree<char>* M = F->insertChild('M');

    GeneralTree<char>* N = G->insertChild('N');

    std::cout << "Tree structure:\n";
    A->printTree();
    std::cout << std::endl;
```

OBSERVATION

I built a template-based GeneralTree class in C++, where each node keeps track of its first child and next sibling instead of a list of children.

Table 9-2

Node	Height	Depth
A	3	0
B	0	1
C	0	1

D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
O	0	3
P	0	3
Q	0	3

Table 9-3

Pre-order	[A,B,C,D,H,E,I,J,P,Q,F,K,L,M,G,N]
In-order	[B,A,C,H,D,I,E,P,J,Q,K,F,L,M,N,G]
Post-order	[B,C,H,D,I,P,Q,J,E,K,L,M,F,N,G,A]

Table 9-4

PREORDER	
CODE	**OUTPUT**

```
//pre order
template <typename T>
void preOrder(GeneralTree<T>* node) {
    if (node == nullptr) return;

    std::cout << node->getRootVal() << " ";

    GeneralTree<T>* child = node->getFirstChild();
    while (child != nullptr) {
        preOrder(child);
        child = child->getNextSibling();
    }
}
```

```
Pre-order: A B C D H E I J P Q F K L M G N
```

OBSERVATION:
The output of the preOrder function, A B C D H E I J P Q F K L M G N confirms that the recursive function correctly visits each node before its children, using getFirstChild() and getNextSibling() to loop across every level of children.

INORDER	
CODE	**OUTPUT**

```
//in order
template <typename T>
void inOrder(GeneralTree<T>* node) {
    if (node == nullptr) return;

    GeneralTree<T>* firstChild = node->getFirstChild();

    if (firstChild != nullptr) {
        inOrder(firstChild);
    }

    std::cout << node->getRootVal() << " ";

    if (firstChild != nullptr) {
        GeneralTree<T>* nextChild = firstChild->getNextSibling();
        while (nextChild != nullptr) {
            inOrder(nextChild);
            nextChild = nextChild->getNextSibling();
        }
    }
}
```

```
In-order: B A C H D I E P J Q K F L M N G
```

OBSERVATION:
The in-order function output, B A C H D I E P J Q K F L M N G verifying that visiting the first child, then the node, then the remaining children works correctly for a general tree. Since a general tree has no fixed left/right structure like a binary tree, this order only makes sense once the first-child/next-sibling links are set up correctly, which the earlier printTree() output already confirmed.

POSTORDER	
CODE	**OUTPUT**
	
OBSERVATION: The post-order output, B C H D I P Q J E K L M F N G A showing that all children are fully processed before their parent is printed. This is also reflected in node A being the very last value printed, since it is the root and has no ancestor to visit after it.	

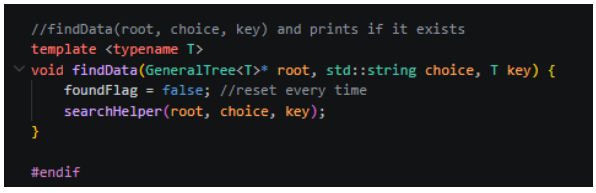
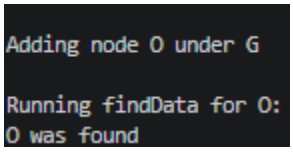
```
//post order
template <typename T>
void postOrder(GeneralTree<T>* node) {
    if (node == nullptr) return;

    GeneralTree<T>* child = node->getFirstChild();
    while (child != nullptr) {
        postOrder(child);
        child = child->getNextSibling();
    }

    std::cout << node->getRootVal() << " ";
}
```

```
Post-order: B C H D I P Q J E K L M F N G A
```

Table 9-6

findData	
CODE	**OUTPUT**
	
OBSERVATION After inserting node O as a child of G, running findData with pre-order traversal successfully located it and printed "O was found," confirming that the search reaches a newly added node without any change to the traversal logic. This shows that findData depends only on the tree's current first-child/next-sibling links rather than on a fixed shape, so it still works correctly after the tree grows.	

```
//findData(root, choice, key) and prints if it exists
template <typename T>
void findData(GeneralTree<T>* root, std::string choice, T key) {
    foundFlag = false; //reset every time
    searchHelper(root, choice, key);
}

#endif
```

```
Adding node O under G

Running findData for O:
O was found
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PREORDER

CODE

```
template <typename T>
void BinarySearchTree<T>::preOrder(){

    std::cout<<key << " ";
    if (left != nullptr){
        left -> preOrder();
    }

    if(right != nullptr){
        right -> preOrder();
    }
}
```

OUTPUT

```
PreOrder : 2 0 1 3 9 4 5 18
```

OBSERVATION:
The BST pre-order output, 2 0 1 3 9 4 5 18, visits the root before either subtree, since preOrder() prints the node first and only then recurses left and right. Because the left subtree of the BST holds every value smaller than the root, the traversal reaches 0 and 1 right after the root, before it ever visits the larger values on the right side.

INORDER

CODE

```
template <typename T>
void BinarySearchTree<T>::InOrder(){
    //go left
    if (left != nullptr){
        left -> InOrder();
    }
    std::cout<< key << " ";
    //go right
    if (right != nullptr){
       right -> InOrder();
    }
}
```

OUTPUT

```
InOrder : 0 1 2 3 4 5 9 18
```

OBSERVATION:
The BST in-order output, 0 1 2 3 4 5 9 18, contains the same values that were inserted but arranged from smallest to largest. This confirms that the binary search tree's ordering rule, where the left child is smaller and the right child is greater than the parent, combined with a left-node-right traversal always produces a sorted sequence.

COMPUTER ENGINEERING

QUEZON CITY

POSTORDER

CODE

```
template<typename T>
void BinarySearchTree<T>::PostOrder()
    //go left
    if (left != nullptr){
        left -> PostOrder();
    }
    //go right
    if (right != nullptr){
       right -> PostOrder();
    }
    std::cout<< key << " ";
}
```

OUTPUT

```
PostOrder: 1 0 5 4 18 9 3 2
```

OBSERVATION:
The BST post-order output, 1 0 5 4 18 9 3 2, ends with the root value 2, since post-order always visits both children before the node itself. This matches the same pattern seen in the general tree's post-order traversal, where the root of any subtree is only printed after everything beneath it has already been visited.

CONCLUSION

In this activity, I learned how to build a general tree using a first-child/next-sibling structure and implement pre-order, in-order, and post-order traversal functions, along with a findData function that reuses those same traversal orders to search the tree. Comparing my by-hand traversals with the actual function outputs helped confirm that my code was working correctly, and the supplementary activity showed how a binary search tree's ordering rule makes its in-order traversal return values in sorted order. I think I did well on this activity since all my outputs matched my expected results, though I could improve on getting the height and depth calculations right on the first try instead of needing a few attempts.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.